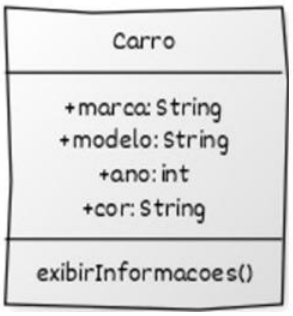
Exercícios de fixação

1. Treinando o conceito de construtor e métodos:

Crie uma classe chamada `Carro` com os seguintes atributos:



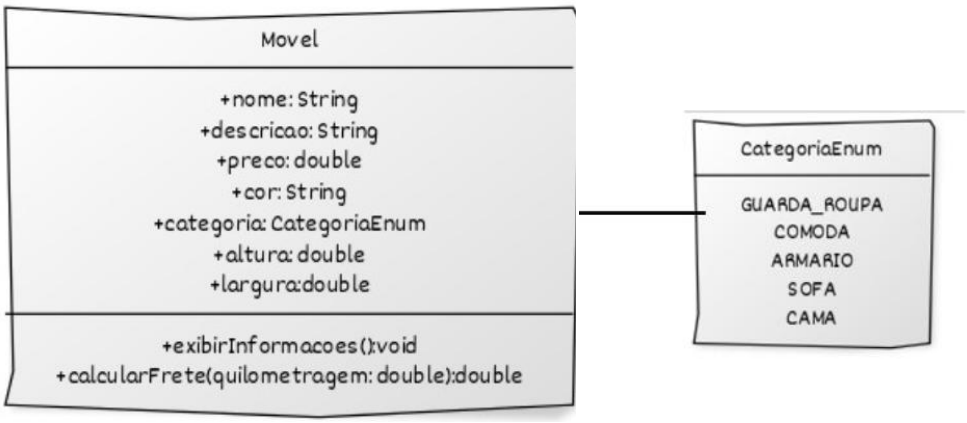
A classe deve ter:

- Um **construtor** que inicializa todos os atributos.
- Um **método** chamado `exibirInformacoes`() que imprime os dados do carro.

No `main`, crie dois objetos dessa classe e chame o método `exibirInformacoes`().

2. Treinando o conceito de enumeração e métodos:

Crie uma classe chamada `Movel` com os seguintes atributos:



A classe deve ter:

- Um **construtor** que inicializa todos os atributos.
- Crie a enumeração CategoriaEnum
- Um **método** chamado `exibirInformacoes`() que imprime os dados do móvel.

Um **método** chamado `calcularFrete`(`quilometragem`: `double`) que calcula o valor do frete aplicando a fórmula: Frete = quilometragem × valor por km + 3% do preço do móvel, onde o valor por km é fixo R$ 10,00 por km.

No `main`, crie dois objetos dessa classe e chame o método `exibirInformacoes`(), e mostre o valor do frete.

3. Treinando o conceito de enumeração, métodos e encapsulamento:

Você foi contratado para desenvolver uma aplicação Java que calcula a quantidade de carne e bebidas necessárias para um churrasco, considerando o número de pessoas e suas preferências.

Para isso, você deve utilizar os conceitos de **classes, atributos, métodos, enumeração, construtor, encapsulamento e relacionamento de associação**.

Requisitos do Sistema

1. **Regras de Cálculo:**

- **Adulto (com carne e bebida)**: 400g de carne + 6 latas de bebida.
- **Adulto (somente carne)**: 400g de carne.
- **Criança**: 200g de carne (não consome bebida).

2. **Classes e Relacionamentos:**

- Pessoa: Representa uma pessoa no churrasco, contendo os atributos:
  - nome (String)
  - idade (int)
  - preferencia (do tipo Preferencia)
- Preferencia (enumeração):
  - CARNE_E_BEBIDA
  - APENAS_CARNE
  - CRIANCA
- Churrasco: Responsável pelo cálculo, contendo:
  - Método calcularConsumo(), que retorna a quantidade total de carne e bebidas necessárias.

3. **Fluxo da Aplicação:**

- O usuário informa a quantidade de pessoas e suas preferências.
- O sistema calcula e exibe a quantidade total de carne e bebidas necessárias

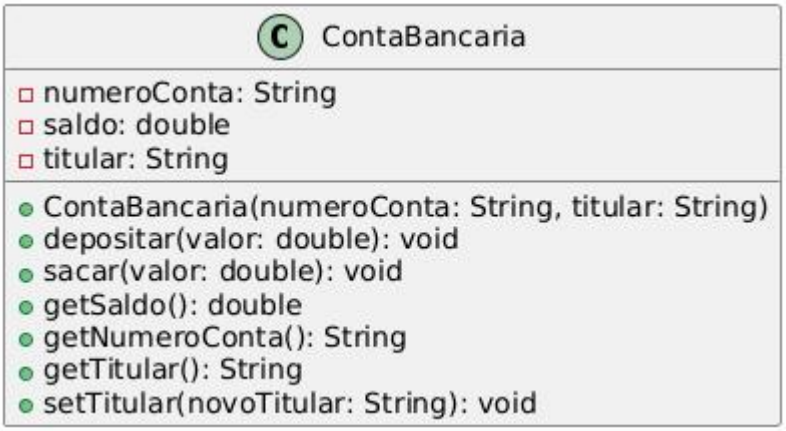
Total de carne necessária: 3.2 kg

Total de bebidas necessárias: 6 latas

Gerenciamento de Conta Bancária

Objetivo: Praticar o encapsulamento básico e a criação de métodos para manipular atributos privados.

Descrição: Crie uma classe ContaBancaria que represente uma conta de banco.



Métodos:

- depositar(double valor): Adiciona um valor ao saldo.
- sacar(double valor): Retira um valor do saldo, se houver saldo suficiente.
- getSaldo(): Retorna o saldo atual.
- getNumeroConta(): Retorna o número da conta.
- getTitular(): Retorna o titular da conta.
- setTitular(String novoTitular): Altera o nome do titular.

Regras de Encapsulamento:

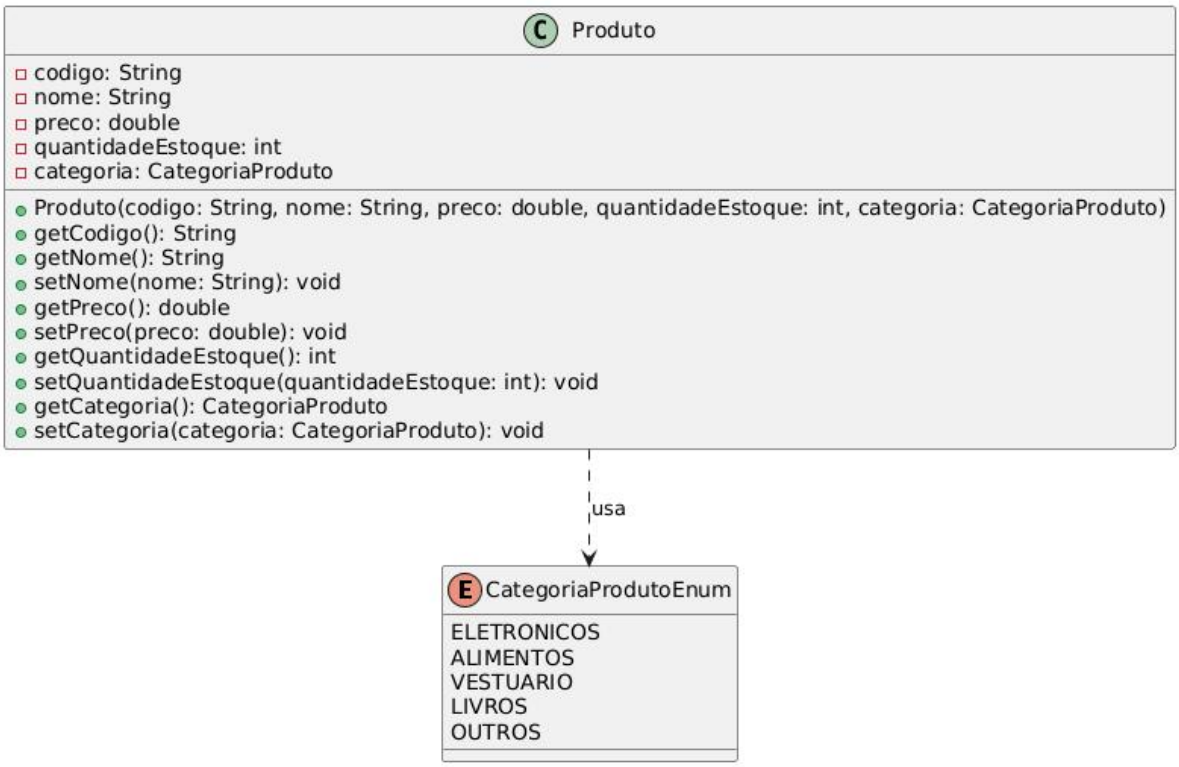
- Todos os atributos devem ser private.
- O saldo só pode ser alterado pelos métodos depositar e sacar.
- O numeroConta não deve ter um método set público, pois não deve ser alterado após a criação da conta.
- No método depositar, o valor deve ser positivo. Se não for, imprima uma mensagem de erro.
- No método sacar, o valor deve ser positivo e não pode exceder o saldo atual. Se alguma dessas condições não for atendida, imprima uma mensagem de erro.

Crie uma classe de teste e instancie objetos para testar todos os métodos.

Cadastro de Produto com Enumeração

Objetivo: Utilizar enumerações para tipos de atributos e aplicar encapsulamento com validação em setters.

Descrição: Crie uma classe Produto para gerenciar informações de produtos em um estoque.



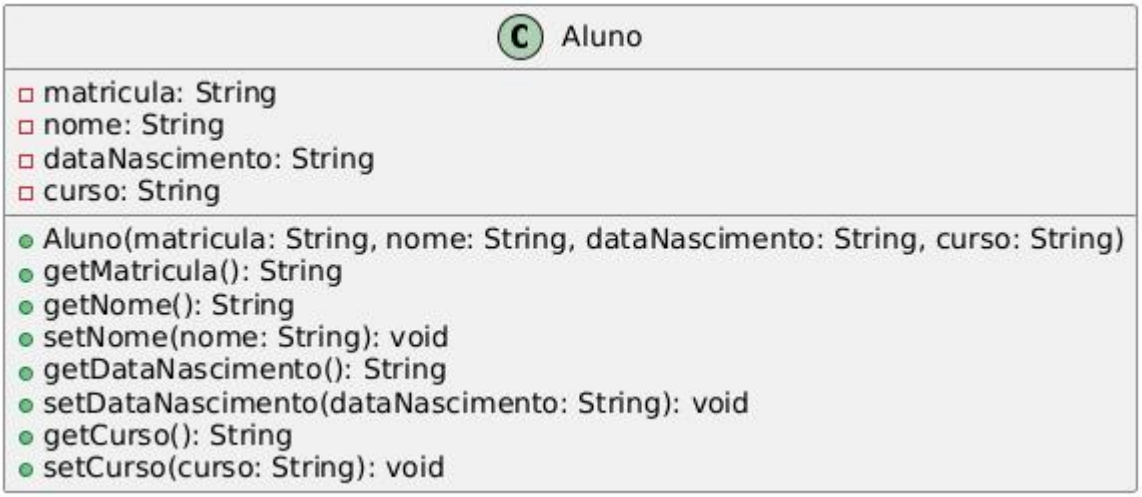
Regras de Encapsulamento:

- Todos os atributos devem ser private.
- O codigo não deve ter um método set público.
- No setPreco, o preço deve ser maior que zero. Caso contrário, imprima uma mensagem de erro e não altere o valor.
- No setQuantidadeEstoque, a quantidade não pode ser negativa. Caso contrário, imprima uma mensagem de erro e não altere o valor.
- No setCategoria, a categoria deve ser uma das definidas na enumeração.

Registro de Aluno com Data de Nascimento

Objetivo: Praticar encapsulamento com validação de dados mais complexa (idade mínima) e uso de um objeto para data.

Descrição: Crie uma classe Aluno para registrar informações de estudantes.



Regras de Encapsulamento:

- Todos os atributos devem ser private.
- A matricula não deve ter um método set público.
- No setDataNascimento, valide se o aluno tem pelo menos 16 anos. Se não tiver, imprima uma mensagem de erro e não altere a data. (Para simplificar, você pode fazer uma validação básica do ano, por exemplo, Integer.parseInt(dataNascimento.substring(6)) <= (AnoAtual - 16)).
- No setNome, o nome não pode ser vazio ou nulo.

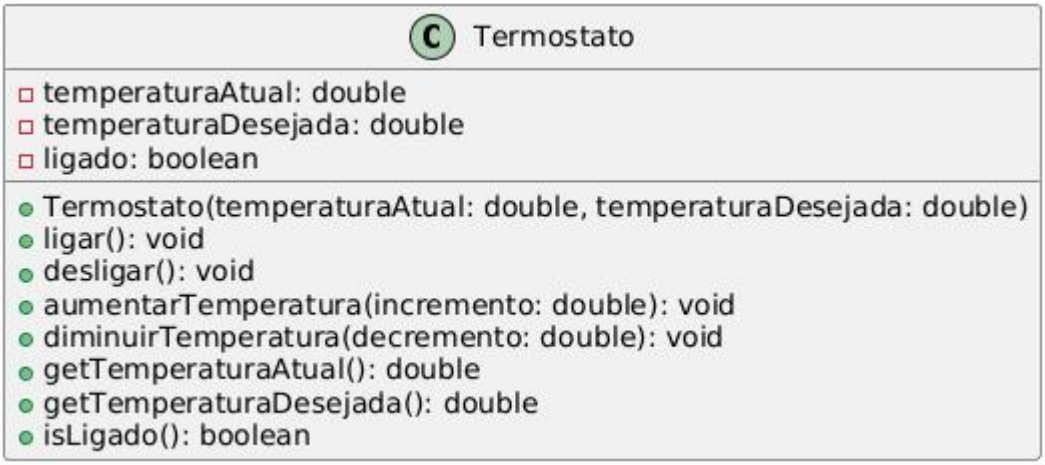
Versão 2

Identifique um atributo que poderia ser uma classe e faça a correção.

Controle de Temperatura de um Termostato

Objetivo: Focar em métodos que controlam o estado interno de um objeto de forma encapsulada, sem expor diretamente os atributos.

Descrição: Crie uma classe Termostato para simular o controle de temperatura.



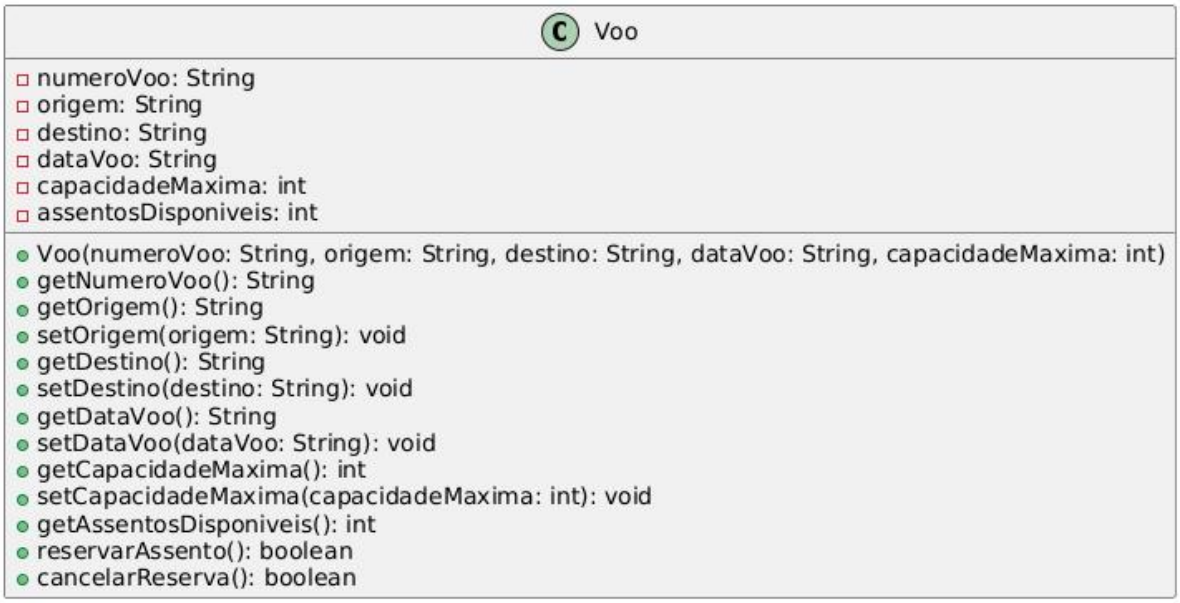
Regras de Encapsulamento:

- Todos os atributos devem ser private.
- Não deve haver setters públicos para temperaturaAtual, temperaturaDesejada ou ligado. A alteração desses estados deve ser feita exclusivamente pelos métodos específicos (ligar, desligar, aumentarTemperatura, diminuirTemperatura).
- A temperaturaDesejada deve estar sempre entre 15.0 e 30.0 graus Celsius. Se aumentarTemperatura ou diminuirTemperatura tentar definir um valor fora desse intervalo, o valor deve ser ajustado para o limite mínimo ou máximo, e uma mensagem de aviso deve ser impressa.
- Os métodos aumentarTemperatura e diminuirTemperatura só devem funcionar se o termostato estiver ligado.

Sistema de Reserva de Voo

Objetivo: Integrar múltiplos conceitos (classes, atributos, métodos, construtores, encapsulamento) em um cenário mais complexo, com validação cruzada de atributos.

Descrição: Crie uma classe Voo para representar um voo em um sistema de reservas.



Construtor:

- Um construtor que inicialize numeroVoo, origem, destino, dataVoo e capacidadeMaxima. assentosDisponiveis deve ser inicializado com o mesmo valor de capacidadeMaxima.

Métodos:

- Getters para todos os atributos.
- Setters para origem, destino, dataVoo e capacidadeMaxima.
- reservarAssento(): Tenta reservar um assento. Se houver disponibilidade, decrementa assentosDisponiveis e retorna true. Caso contrário, retorna false.
- cancelarReserva(): Incrementa assentosDisponiveis se for possível (ou seja, se assentosDisponiveis for menor que capacidadeMaxima) e retorna true. Caso contrário, retorna false.

Regras de Encapsulamento:

- Todos os atributos devem ser private.
- O numeroVoo não deve ter um método set público.
- No setOrigem e setDestino, os valores não podem ser nulos ou vazios, e origem não pode ser igual a destino. Se houver violação, imprima uma mensagem de erro e não altere o valor.

- No setDataVoo, a data deve ser uma data futura em relação à data atual. (Para simplificar, você pode comparar apenas os anos ou usar uma biblioteca de datas se preferir).
- No setCapacidadeMaxima, a nova capacidade deve ser maior ou igual ao número de assentos já ocupados (capacidadeMaxima - assentosDisponiveis). Se a nova capacidade for menor que os assentos ocupados, imprima uma mensagem de erro e não altere. Além disso, a capacidade máxima deve ser um valor positivo.
- Os métodos reservarAssento e cancelarReserva são os únicos responsáveis por alterar assentosDisponiveis.

exercicioIMC > exercicio.txt

```
1)  Criar uma pasta chamada 04ExercicioIMC

2)  Acessar a pasta via terminal: cd 04ExercicioIMC

3)  Criar um projeto via terminal: dotnet new console

4)  Criar uma classe chamada Pessoa

5)  Na classe pessoa adicionar os atributos: peso e altura

6)  Criar um método para retornar o IMC, será um método double
com o cálculo: peso / (altura * altura)

7)  Criar um método para retornar um string dependendo do IMC:
<  18.5 - Abaixo do peso
<  25   - Peso normal
<  30   - Acima do peso
<  35   - Obesidade I
<  40   - Obesidade II
>= 40   - Obesidade III

8)  Criar um método de mensagem para exibir o cálculo e a situação

9)  Instânciar um objeto na classe Program
```